从论文 Section 3 到 Athena++ 模拟

Svanberg, Cui & Latter (2022) · Figure 2 实现说明

1. 目标与代码分工

本说明回答：如何把论文 Section 3 的模型转成可运行的 Athena++ 问题设置。目标是 Figure 2 的 $t = 500P_0$ 快照，而非先运行到整篇论文后续分析所用的 $3000P_0$。Figure 2 左图展示 $R \in [1, 2]$ 的 v_z/c_s，右图展示 $R \in [1.7, 2]$ 的子午面速度。

流体演化由 Athena++ 求解；本仓库负责问题初始化、平衡边界、快速冷却、参数和后处理。 我们以 Athena++ 自带的 `disk.cpp` 为基础建立独立的 `svanberg.cpp`，保留其平衡盘与边界框架，再加入确定性速度扰动和论文式 (27) 的冷却。没有重新编写 Godunov 求解器，也没有预先指定波纹形状。

- 上游求解器：PrincetonUniversity/athena。
- 本仓库：cc795/course_VSI。
- 本次实现固定的 Athena++ 版本：`053bc738d5c85dac717326b4338b34c7db4e715d`。
- 核心文件：`src/svanberg.cpp`、`inputs/athinput.figure2`。

本说明以实际代码为依据，将“论文明确给定”与“为可重复计算而补充选择”分开说明。它说明实现方法，不代表 Figure 2 的目标时刻已经完成或经过收敛验证。

2. Section 3.1：实际求解什么方程

论文式 (19)–(21) 是含中心引力与热松弛的欧拉方程：

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) = 0,$$

$$\frac{\partial (\rho \mathbf{v})}{\partial t} + \nabla \cdot (\rho \mathbf{v}\mathbf{v} + P\mathbf{I}) = -\rho \nabla \Phi,$$

$$\frac{\partial E}{\partial t} + \nabla \cdot [(E + P)\mathbf{v}] = -\rho \mathbf{v} \cdot \nabla \Phi - \Lambda_c.$$

其中

$$E = \epsilon + \tfrac{1}{2}\rho |\mathbf{v}|^2, \qquad P = (\gamma - 1)\epsilon, \qquad \gamma = \frac{7}{5}, \qquad \Phi = -\frac{GM}{r}.$$

编译选择为 `--coord=spherical_polar --eos=adiabatic --flux=hllc -omp`，不启用磁场。`nx3=1` 表示轴对称：不解析方位角变化，但仍演化三个速度分量，尤其保留盘的旋转 v_ϕ。

输入中的 `problem/GM=1` 启用 Athena++ 内置点质量引力源项，处理动量中的引力及能量中的引力功。问题生成器中不再重复添加同一引力。这里的能量方程包含冷却源项，因此不能要求气体总能量在热松弛时保持不变。

为何不用全局等温状态方程？ 论文的温度随柱半径变化，需要局部等温目标 $c_s(R)$。本实现保留绝热能量方程，再用极快的热松弛逼近该目标，而不是使用一个空间恒定的等温声速。

3. Section 3.2：从解析平衡建立初态

3.1 单位、温度和密度幂律

取 $GM = R_0 = 1$，并选密度归一化 $\rho_0 = 1$。在忽略自引力的该模型中，后者用于设定密度单位。内边界参考开普勒频率和周期为

$$\Omega_0 = \sqrt{GM/R_0^3} = 1, \qquad P_0 = 2\pi.$$

此参考周期按开普勒频率定义，并非用包含压强支撑后的初始气体角速度重新定义。注意代码中的 `orbital_advection/Omega0` 表示参考系的旋转角速度，默认取 0；它与这里定义周期的物理参考频率 $\Omega_0 = 1$ 不是同一个参数。

论文式 (22)、(23) 给出

$$\rho_{\rm mid} = \rho_0 (R/R_0)^{-q_D}, \qquad c_s^2 = c_0^2 (R/R_0)^{-q_T}.$$

取 $q_D = 2$、$q_T = 1$、$h = H/R = 0.05$。由 $H = c_s/\Omega_K$ 得

$$c_0 = h\sqrt{GM/R_0} = 0.05, \qquad c_s^2(R) = 0.0025/R.$$

代码变量名称沿用上游 `disk.cpp`，指数已经包含负号：

物理量	输入参数与取值
密度指数 $-q_D$	`dslope = -2.0`
声速平方指数 $-q_T$	`pslope = -1.0`
c_0^2	`p0_over_r0 = 0.0025`
ρ_0	`rho0 = 1.0`
γ	`hydro/gamma = 1.4`

`PoverR()` 返回的是平衡目标 $P/\rho = c_s^2(R)$，而不是 c_s 本身。

3.2 垂直密度平衡：式 (24)

每个单元先由球坐标计算柱坐标：

$$R = r\sin\theta, \qquad z = r\cos\theta.$$

随后使用论文完整势函数下的密度解，而非仅用薄盘高斯近似：

$$\rho(R,z) = \rho_0 (R/R_0)^{-q_D} \exp\left[\frac{GM}{c_s^2(R)}\left(\frac{1}{r} - \frac{1}{R}\right)\right].$$

这对应 `DenProfileCyl()`。远离中面时 $r > R$，指数为负，密度随高度下降。压强取 $P = \rho c_s^2(R)$。

式 (24) 对应的实际 C++ 代码

下面摘录 `src/svanberg.cpp` 中的真实函数，仅调整较长行的换行。函数中的 `rad` 是柱半径 R，不是球半径 r。

```
Real PoverR(const Real rad, const Real phi, const Real z) {
  Real poverr;
  poverr = p0_over_r0*std::pow(rad/r0, pslope);
  return poverr;
}
```

```
Real DenProfileCyl(const Real rad, const Real phi, const Real z) {
  Real den;
  Real p_over_r = p0_over_r0;
  if (NON_BAROTROPIC_EOS)
    p_over_r = PoverR(rad, phi, z);
  Real denmid = rho0*std::pow(rad/r0,dslope);
  Real dentem = denmid*std::exp(
      gm0/p_over_r*(1./std::sqrt(SQR(rad)+SQR(z))-1./rad));
  den = dentem;
  return std::max(den,dfloor);
}
```

代码表达式	对应的数学量
`PoverR(rad, phi, z)`	$c_s^2(R) = c_0^2(R/R_0)^{-q_T}$
`rho0*pow(rad/r0,dslope)`	$\rho_0(R/R_0)^{-q_D}$，即中面密度
`sqrt(SQR(rad)+SQR(z))`	$\sqrt{R^2+z^2} = r$，球半径
`gm0/p_over_r`	$GM/c_s^2(R)$
`1./sqrt(...)-1./rad`	$1/r - 1/R$
`denmid*exp(...)`	式 (24) 的完整密度

这里 `dslope=-2`、`pslope=-1`，因此代码中的指数已经包含论文负号。`std::max(den,dfloor)` 是额外的数值密度底限；只有解析密度小于底限时才偏离式 (24)。

`NON_BAROTROPIC_EOS` 在本次绝热能量方程编译中成立，所以实际执行的是随 R 变化的 `PoverR()` 分支。`phi` 保留在函数接口中，但这个轴对称平衡解不依赖它。

3.3 旋转平衡与垂直剪切：式 (25)

定义 $\Omega_K(R) = \sqrt{GM/R^3}$。论文的平衡角速度为

$$\Omega(R,z) = \Omega_K(R)\left[-(q_D+q_T)h^2 + (1-q_T) + q_T\frac{R}{r}\right]^{1/2}.$$

`VelProfileCyl()` 返回的是线速度 $v_\phi = R\Omega$。代入本次参数：

$$v_\phi = \sqrt{GM/R}\,\sqrt{-3h^2 + R/r}.$$

该式同时包含压强支撑和随高度变化的旋转速度。不能用纯开普勒速度 $\sqrt{GM/R}$ 替代，否则会改变论文需要的垂直剪切背景。论文式 (26) 的薄盘近似也表明 $\partial\Omega/\partial z \neq 0$。

`ProblemGenerator()` 先设置平衡密度和三个动量分量，再加入噪声，最后由**包含噪声后的总动能**构造 E。平衡状态中 $v_r = v_\theta = 0$；它们不是在演化过程中被强制设为零。

式 (25) 对应的实际 C++ 代码

```
Real VelProfileCyl(const Real rad, const Real phi, const Real z) {
  Real p_over_r = PoverR(rad, phi, z);
  Real vel = (dslope+pslope)*p_over_r/(gm0/rad)
             + (1.0+pslope)
             - pslope*rad/std::sqrt(rad*rad+z*z);
  vel = std::sqrt(gm0/rad)*std::sqrt(vel) - rad*Omega0;
  return vel;
}
```

第一条给 `vel` 赋值的语句计算的是**无量纲括号**，还不是速度。它与式 (25) 的三项逐项对应：

$$\begin{aligned}(\mathrm{dslope}+\mathrm{pslope})\frac{\mathrm{p_over_r}}{\mathrm{gm0/rad}} &= -(q_D+q_T)h^2,\\ 1+\mathrm{pslope} &= 1-q_T,\\ -\mathrm{pslope}\,\frac{\mathrm{rad}}{\sqrt{\mathrm{rad}^2+z^2}} &= q_T\frac{R}{r}.\end{aligned}$$

第一行等式使用

$$h^2=\frac{c_s^2}{(R\Omega_K)^2}=\frac{c_s^2}{GM/R}.$$

第二条给 `vel` 赋值的语句再乘 $\sqrt{GM/R}=R\Omega_K$，因此返回的是 $v_\phi=R\Omega$，不是角速度 Ω。尾部的 `-rad*Omega0` 适配旋转参考系；本次该参数为 0，所以不改变论文的惯性系速度。

初始化时，`ProblemGenerator()` 将该速度写入球坐标方位动量：

```
vel = VelProfileCyl(rad,phi,z);
// ... spherical-polar branch ...
phydro->u(IM3,k,j,i) = den*vel;
```

`IM3` 保存的是 ρv_ϕ，不能直接把角速度或线速度不乘密度就写入该位置。四个平衡边界函数也调用同一个 `VelProfileCyl()`，使初态与边界采用相同的旋转平衡。

4. Section 3.2：实现式 (27) 的快速冷却

论文使用

$$\Delta T=(T_0-T)\left[1-\exp(-\Delta t/\tau)\right],\qquad \tau=10^{-10}P_0.$$

因此输入中的代码单位冷却时间为

$$\tau=2\pi\times10^{-10}=6.283185307179586\times10^{-10}.$$

`MeshBlock::UserWorkInLoop()` 在每个完整时间步之后执行一次热松弛。设单元动量为 $\mathbf{m}=\rho\mathbf{v}$：

$$K=\frac{|\mathbf{m}|^2}{2\rho},\qquad \epsilon=E-K,\qquad \epsilon_0=\frac{\rho c_s^2(R)}{\gamma-1},$$

$$f=1-e^{-\Delta t/\tau},\qquad \epsilon_{\mathrm{new}}=\epsilon+f(\epsilon_0-\epsilon),$$

$$E_{\mathrm{new}}=K+\epsilon_{\mathrm{new}},\qquad P_{\mathrm{new}}=(\gamma-1)\epsilon_{\mathrm{new}}.$$

在这个子步骤中保持密度和动量不变，只改变内能；并同步更新 Athena++ 的守恒能量和原始压强。代码通过 `-expm1(-dt/tau)` 计算 f。

这是一种按完整步进行的算子分裂更新。由于 τ 远小于流体时间步，通常 f 在机器精度下接近 1，温度迅速回到径向指定的目标。不能直接用未经处理的显式 $\Delta t/\tau$ 冷却项代替，否则会引入极强的刚性和稳定性问题。

快速冷却抑制会阻碍 VSI 的热浮力响应，但实际出现的波和非线性演化仍由流体方程产生，冷却函数没有植入波纹。

4.1 式 (27) 对应的实际 C++ 代码

下面完整列出实际冷却函数，循环与长表达式仅作排版换行。Athena++ 每个完整时间步调用该函数一次，而不是在这里推进一个独立的流体时间步。

```
void MeshBlock::UserWorkInLoop() {
  const Real f=-std::expm1(-pmy_mesh->dt/tau);
  for(int k=ks;k<=ke;++k)
    for(int j=js;j<=je;++j)
      for(int i=is;i<=ie;++i) {
        const Real rho=phydro->u(IDN,k,j,i);
        const Real kinetic=0.5*(SQR(phydro->u(IM1,k,j,i))+
          SQR(phydro->u(IM2,k,j,i))+
          SQR(phydro->u(IM3,k,j,i)))/rho;
        Real R,phi,z;
        GetCylCoord(pcoord,R,phi,z,i,j,k);
        const Real target=rho*PoverR(R,phi,z)/(gamma_gas-1.0);
        const Real internal=phydro->u(IEN,k,j,i)-kinetic;
        const Real cooled=internal+f*(target-internal);
        phydro->u(IEN,k,j,i)=kinetic+cooled;
        phydro->w(IPR,k,j,i)=(gamma_gas-1.0)*cooled;
      }
}
```

数组里保存的物理量

数组项或变量	含义
`u(IDN)`	密度 ρ
`u(IM1)`, `u(IM2)`, `u(IM3)`	动量密度 $\rho v_r, \rho v_\theta, \rho v_\phi$
`u(IEN)`	总能量密度 E
`w(IPR)`	原始变量中的压强 P
`kinetic`	动能密度 K，不是单位质量动能
`target`	目标内能密度 ϵ_0，不是温度或压强
`internal` / `cooled`	冷却前 / 后的内能密度

逐条对应式 (27)

1. `-expm1(-dt/tau)` 等于 $1-\exp(-\Delta t/\tau)$，就是论文的松弛系数。使用 $\mathrm{expm1}(x)=e^x-1$ 可减轻小参数时的相减损失。
2. 三个 `u(IM*)` 是动量，因此先求平方和，再除以密度并乘 $1/2$，得到 $K=|\mathbf{m}|^2/(2\rho)$。
3. `target` 用当前单元密度、径向目标 c_s^2 和 $\gamma-1$ 计算 $\epsilon_0=\rho c_s^2/(\gamma-1)$。
4. `internal=E-K` 从总能量中分离出当前内能。若直接把总能量松弛到 `target`，就会错误地消掉旋转动能。
5. `cooled=internal+f*(target-internal)` 对内能应用与论文温度更新相同的指数松弛。
6. `u(IEN)=kinetic+cooled` 把原动能加回；`w(IPR)=(gamma-1)*cooled` 同步更新压强，使后续时间步估计读取一致的热状态。

为什么更新内能等价于更新温度？在这个冷却子步骤中 ρ 不变、分子量固定，且 $\epsilon\propto\rho T$。因此把论文的温度增量乘上同一个比例系数，就得到代码中的内能增量；密度和三个动量在该函数中均未被重新赋值。

与输入文件的连接

```
<hydro>
gamma = 1.4
<problem>
p0_over_r0 = 0.0025
pslope = -1.0
dslope = -2.0
tau = 6.283185307179586e-10
```

`InitUserMeshData()` 读取这些参数，`PoverR()` 将前两项组合成 $c_s^2(R)$，而每一步冷却都会重新在当前单元位置计算目标内能。参数 `tau` 已经换成代码时间，不能直接误填为没有乘 2π 的 `1e-10`。

5. Section 3.3：区域、网格、扰动与边界

5.1 完整计算域与 Figure 2 裁剪区

设置	本实现
球半径	$r \in [1, 30]$
极角	$\theta \in [\pi/2 - 0.25, \pi/2 + 0.25]$
网格	$704 \times 448 \times 1$
径向网格参数	`x1rat=1.005`
极角网格	均匀
Figure 2 目标	$500P_0 = 3141.592653589793$ 代码时间

Figure 2 只显示内盘，但模拟保留整个 $r \in [1, 30]$ 区域，不能把作图范围直接当作计算域。球坐标中的边界和柱坐标图像的边界也不是同一组直线。

论文网格描述的两处需要说明。 第一，极角区间长度为 $10h = 0.5$，448 格给出中面附近约 $h/(0.5/448) = 44.8$ 格每尺度高，与文字中的约 12 格/H 不一致。这里优先采用明确写出的范围和单元数。第二，若严格采用 $\Delta r \propto r$，704 格覆盖 $[1, 30]$ 的比例应为 $30^{1/704} \simeq 1.004843$。本实现优先采用论文给出的相邻宽度比 1.005，由 Athena++ 固定区间端点并几何递增单元宽度。因此这是所记录的网格解释，不应宣称能由论文文字唯一确定原作者的全部网格细节。

5.2 初始扰动

论文规定速度噪声为局部声速的 1%，没有给出分布、种子或每个分量的具体构造。本实现选择

$$\delta v_a = 0.01 c_s \, \xi_a, \qquad \xi_a \sim U[-1, 1], \quad a \in \{r, \theta, \phi\}.$$

`Noise()` 用坐标和分量编号作确定性散列，种子为 `202211`。这是为了重复计算和改变分块时保持同一初态；不能把该种子说成论文给定。

5.3 边界与并行分块

径向和极角的内、外边界使用四个 `Disk*X1/X2()` 函数，在幽灵单元恢复平衡 ρ、P 和 v_ϕ，并令平衡的径向、极角速度为零。边界不持续注入初始随机噪声，也不额外增加论文未指定的阻尼层。方位角边界为周期条件，因 `nx3=1` 不解析方位角模式。

默认将区域分为四个 176×448 网格块，使用四个 OpenMP 线程；这是计算组织方式，不是额外的物理参数。初始密度底限 `dfloor=1e-12`、压强底限 `pfloor=1e-16` 属于数值保护设置，不是论文明确列出的物理参数。

5.4 Section 3.3 原文逐项对应代码

这段 setup 并不全部位于一个 C++ 函数里。**范围、单元数、边界类型和终止时间由输入文件给出；初始噪声和平衡边界由问题生成器执行；实际网格和时间推进由 Athena++ 内核完成。**

原文要求	对应文件与位置
$r \in [1, 30]$，宽度比 1.005	输入 `<mesh>` 的 `x1min/x1max/x1rat`
$\theta \in [\pi/2 - 5h, \pi/2 + 5h]$	输入 `<mesh>` 的 `x2min/x2max`
704×448，二维	输入 `nx1/nx2/nx3`；球坐标编译选项
$GM = R_0 = 1$	输入 `<problem>` 的 `GM/r0`；初始化函数读取
$3000P_0$ 与 $P_0 = 2\pi/\Omega_0$	输入 `<time>` 的 `tlim`；见下文时间区别
1% 局部声速噪声	`ProblemGenerator()` 中三条动量增量语句
边界严格恢复平衡	`<mesh>` 的 `user` 标志及四个边界函数

A. 径向范围、极角范围与单元数

以下为 `inputs/athinput.figure2` 的真实 `<mesh>` 设置，省略不相关行：

```
<mesh>
nx1 = 704
x1min = 1.0
x1max = 30.0
x1rat = 1.005
ix1_bc = user
ox1_bc = user
nx2 = 448
x2min = 1.3207963267948966
x2max = 1.8207963267948966
ix2_bc = user
ox2_bc = user
nx3 = 1
```

在 `spherical_polar` 坐标下，`x1` 就是球半径 r，`x2` 就是极角 θ。将 $h = 0.05$ 代入论文范围：

$$\frac{\pi}{2} - 5h = 1.3207963267948966, \qquad \frac{\pi}{2} + 5h = 1.8207963267948966.$$

因此没有把极角范围作为另一自由参数。输入没有显式指定 `x2rat`，Athena++ 默认取 1，即均匀极角网格。`nx3=1` 与球坐标编译选项一起给出轴对称二维计算。

`x1rat=1.005` 交给 Athena++ 内核生成真实单元面位置。对应上游 `src/mesh/mesh.hpp` 中的 `DefaultMeshGeneratorX1()`，而不是本仓库 `ProblemGenerator()` 中的随机噪声函数。对本次参数，其单元面可写为

$$r_i = 1 + 29\frac{q^i - 1}{q^{704} - 1}, \qquad q = 1.005, \quad i = 0, \ldots, 704.$$

这个表达式既保持两个端点，也使 $\Delta r_{i+1}/\Delta r_i = q$。单元数和范围确定后，前文提到的每尺度高分辨率估算不一致就需要单独报告，不能再另设一个“12 cells/H”开关来同时满足相互矛盾的数字。

B. 单位与终止时间

输入文件中：

```
<problem>
GM = 1.0
r0 = 1.0
rho0 = 1.0
```

`InitUserMeshData()` 对应读取：

```
gm0 = pin->GetOrAddReal("problem","GM",0.0);
r0 = pin->GetOrAddReal("problem","r0",1.0);
rho0 = pin->GetReal("problem","rho0");
```

`gm0`、`r0` 随后用于密度和旋转解析解；同一个输入 `GM` 也由 Athena++ 内置引力模块读取。

论文 setup 的总时长与当前输入并非同一个目标。 论文说运行到 $3000P_0$，其代码时间为

$$t_{3000} = 3000 \times 2\pi = 18849.55592153876.$$

用户本次首先要求 Figure 2，该图时间为 $500P_0$，所以实际输入为:

```
<time>
nlim = -1
tlim = 3141.592653589793
```

`nlim=-1` 表示不设置步数上限，`tlim=500\times2\pi` 控制目标终止时间。不能声称当前输入已经实现论文整段 $3000P_0$ 的分析时长。如果后续要改为该完整时长，输入与运行脚本/续算命令中的时间覆盖项都需要一致修改；仅改变 PDF 中的数字不会改变模拟。本次文档更新没有修改正在运行的目标。

C. 1% 声速噪声如何加入守恒量

输入设置:

```
noise_amp = 0.01
seed = 202211
```

`InitUserMeshData()` 读取幅度和种子，`ProblemGenerator()` 在平衡动量上加入:

```
Real cs=std::sqrt(PoverR(rad,phi,z));
phydro->u(IM1,k,j,i)+=den*noise_amp*cs*Noise(x1,x2,0);
phydro->u(IM2,k,j,i)+=den*noise_amp*cs*Noise(x1,x2,1);
phydro->u(IM3,k,j,i)+=den*noise_amp*cs*Noise(x1,x2,2);
```

这里 `cs` 随柱半径变化，因此确实是局部声速的 1%。守恒数组保存 $\rho\mathbf{v}$，故增量为 $\rho\delta\mathbf{v}$，代码中的 `den*` 不能省掉。`Noise()` 返回 $[-1,1)$ 内的确定性伪随机值；最后一个参数区分三个速度分量。

这些语句只在初始化时执行，不是在每一步持续驱动。噪声加入后，初始总能量也按含噪声的三个动量重新计算，避免动量和能量不一致。

D. “边界严格为平衡解”如何执行

第一层是输入文件中的四个 `user` 边界标志。第二层是在 `InitUserMeshData()` 注册实际函数，例如:

```
if (mesh_bcs[BoundaryFace::inner_x2] == GetBoundaryFlag("user")) {
  EnrollUserBoundaryFunction(BoundaryFace::inner_x2, DiskInnerX2);
}
```

其他三个方向分别注册 `DiskInnerX1`、`DiskOuterX1` 和 `DiskOuterX2`。第三层是函数真正给幽灵单元赋值。以 `DiskInnerX2()` 的球坐标分支为例，下面是实际循环体中的核心语句:

```
GetCylCoord(pco,rad,phi,z,i,jl-j,k);
prim(IDN,k,jl-j,i) = DenProfileCyl(rad,phi,z);
vel = VelProfileCyl(rad,phi,z);
// Optional orbital-advection adjustment is inactive in this run.
prim(IM1,k,jl-j,i) = 0.0;
prim(IM2,k,jl-j,i) = 0.0;
prim(IM3,k,jl-j,i) = vel;
if (NON_BAROTROPIC_EOS)
  prim(IEN,k,jl-j,i) = PoverR(rad,phi,z)*prim(IDN,k,jl-j,i);
```

`jl-j` 定位极角内边界的幽灵层。赋值依次恢复式 (24) 的密度、零子午面速度、式 (25) 的旋转线速度和 $P = \rho c_s^2$。这一段操作的是**原始变量数组 `prim`**，所以速度不乘密度。上游枚举 `IEN=IPR=4`；该数组里 `prim(IEN)` 使用的是压强槽位，不是把总能量赋成了压强。

四个边界均使用相同的平衡函数，所以与初态一致；不往边界重新加入初始化噪声。只恢复幽灵单元，不把相邻活动单元全部锁死为平衡状态。活动区域内的波由流体通量演化，这也是边界条件与人为强制整个盘保持静态的区别。

6. 从输出到 Figure 2 与 Movie

Athena++ 输出球坐标正交速度分量。后处理在每个单元中心转换为

$$v_z = v_r \cos\theta - v_\theta \sin\theta, \qquad v_R = v_r \sin\theta + v_\theta \cos\theta.$$

左图的颜色值为

$$\frac{v_z}{c_s(R)}, \qquad c_s(R) = \sqrt{0.0025/R}.$$

这里使用平衡目标的**局部等温声速**，不额外乘上 $\sqrt{\gamma}$。由于 v_z 有正有负，不能直接取普通对数；绘图使用带线性零点附近区间的对称对数归一化，保留速度方向。

静态双面板函数在主图显示 $R \in [1, 2]$，并在右侧显示 $R \in [1.7, 2]$ 的速度矢量。按用户要求制作的 Movie 则删除右侧面板，采用深色底、青蓝—正红配色和 Times 风格字体，标题显示整数轨道时刻；这些都是展示选择，不改变数值数据。所有帧使用相同的颜色范围和空间坐标范围。

每 $1P_0$ 保存一次完整快照、每 $10P_0$ 保存重启、每 $0.1P_0$ 保存 history，属于本项目选择的输出频率。常规输出写在首次达到或超过计划时间的时间步；标题的整数 P_0 是名义输出时刻，准确时间记录在随视频提供的元数据中。Movie 每个快照停留 0.25 秒，不在不同快照间生成虚构的插值流场。

早期未保存的时刻由同一初态另行补算。补算初始四个分块与主计算逐字节一致；但主计算发生过检查点重启，因此不把两段描述为逐步浮点轨迹完全相同。原始 VTK 是单精度输出，流体求解仍为双精度。

7. 代码索引与复现边界

论文内容	代码入口
参数及边界注册	`Mesh::InitUserMeshData()`
初态、扰动、守恒能量	`MeshBlock::ProblemGenerator()`
球坐标到柱坐标	`GetCylCoord()`
式 (23)：目标 c_s^2	`PoverR()`
式 (24)：平衡密度	`DenProfileCyl()`
式 (25)：旋转线速度	`VelProfileCyl()`
平衡幽灵单元	`Disk*X1/X2()`
式 (27)：热松弛	`MeshBlock::UserWorkInLoop()`
静态图、Movie	`plot_figure2.py`、`make_movie.py`

论文明确给定： 方程、γ、中心势、q_D、q_T、h、区域、网格数量、极短冷却时间、1% 速度噪声、平衡边界，以及 Figure 2 的时间。

本实现补充选择： 现代固定版 Athena++、VL2/PLM/HLLC、CFL=0.3、密度归一化、噪声分布与种子、数值底限、并行分块、输出频率和视觉设计。论文未给出的这些内容不能反向归为作者原始设置。运行和初始化检查也不能代替分辨率收敛检验。

判断是否复现 Figure 2，应在目标时刻检查内盘大尺度、跨中面的 corrugation 振荡，以及速度场结构与幅度；不要求随机波峰的位置逐像素一致。低分辨率测试、早期快照和仅能正常运行的程序，都不足以证明目标图已复现。二维轴对称计算也不能据此证明三维非轴对称次级不稳定性得到了充分解析。Section 3.4 中 2000–3000 P0 的波区和传播分析属于另一阶段，不能从 Figure 2 的单个早期目标快照推断。

参考资料

1. Svanberg, E., Cui, C. & Latter, H. N. (2022). Wavelike nature of the vertical shear instability in global protoplanetary discs. MNRAS **514**, 4581–4587. DOI: 10.1093/mnras/stac1598，重点为 Section 3、式 (19)–(27)、Figure 2。
2. Athena++ 官方源码：github.com/PrincetonUniversity/athena。平衡盘框架参考其 `src/pgen/disk.cpp`。
3. 本项目源码：github.com/cc795/course_VSI。可直接阅读 `src/svanberg.cpp` 与 `inputs/athinput.figure2` 对照本说明。